

Natural Language Processing

CSC 733

Lecture 2: Words, Tokens, and Text

From written language to computational input

Yuji Zhang · College of Staten Island, CUNY

From Lecture 1 to Lecture 2

We move from history to the first technical question.

Last class

History of NLP
Symbolic systems
Statistical NLP
Neural NLP
Large language models



Today

What is the input?
What is a word?
What is a token?
How do we tokenize
text?



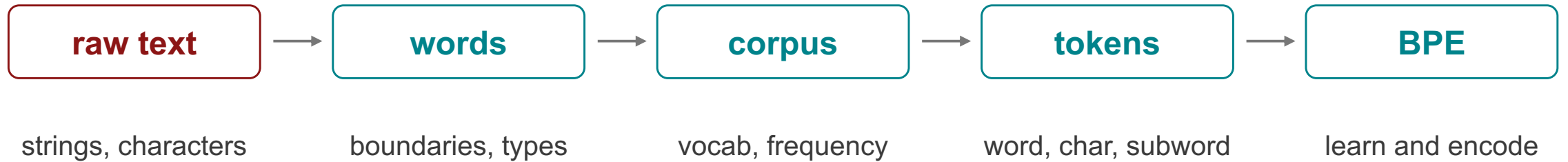
Later

Embeddings
Language models
Structured prediction
Neural networks
Transformers

We do not start from prediction. We start from text.

Today: from text to tokens

A prerequisite-aware map for the whole lecture.



Midterm material starts here.

What does a computer actually see?

A sentence is not automatically a model input.

Humans see:

The movie was surprisingly good.

A program initially sees:

a sequence of characters, bytes, or code points

Question for today:

Where should one computational unit begin and end?

How many words in a sentence?

They picnicked by the pool, then
lay back on the grass and looked
at the stars.

16 words

- if we don't count punctuation marks as words

18 if we count punctuation

Counting depends on a rule

A tokenization rule must be explicit.

Sentence:

They picnicked by the pool, then lay back on the grass.

Rule	Result
Ignore punctuation	11 word tokens
Separate punctuation	13 tokens
Normalize case	type counts may change, e.g., The vs the

The rule determines the count.

Tokens and types in one sentence

Occurrences and vocabulary items are different.

They picnicked by the pool, then lay back on the grass and looked at the stars.

Concept	Meaning	Example
token	one occurrence in running text	the occurs 3 times
type	one distinct item	the is 1 type
vocabulary	the set of types	all distinct units under the rule

If we ignore punctuation: 16 tokens, 14 types.

Why 14 types? The word “the” appears three times but contributes one type.

Type, token, vocabulary

Three terms we will use all semester.

Term	Definition	Example
token	one occurrence in running text	the first “the”
type	unique item in the vocabulary	the
vocabulary V	set of all types known to a system	{the, cat, sat, ...}

Notation: vocabulary size = $|V|$

How many words in a sentence?

I 'm

Orthographically one word (in the English writing system)

But **grammatically** two words:

1. the subject pronoun I
2. the verb 'm, short for am

How many words in an utterance?

"I do uh main- mainly business data processing"

Disfluencies

- Fragments *main-*
- *Filled pauses: uh and um*
- Should we consider these to be words?

How many words in a sentence?

Not every written language uses spaces!!

Chinese, Japanese and Thai don't!

How to choose tokens in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”
yáo míng jìn rù zǒng jué sài

3 words?

姚明 进入 总决赛

YaoMing reaches finals

Chinese Treebank

5 words?

姚 明 进入 总 决赛

Yao Ming reaches overall finals

Peking
University

7 words?

姚 明 进 入 总 决 赛

Yao Ming enter enter overall decision game

Just use characters

Tokenization across languages

No single word-boundary rule works for every language.

- Whitespace tokenization works only when spaces reliably mark the units we want.
- Character-based tokenization can be useful in some Chinese settings, but it is not a universal solution.
- Thai, Japanese, Chinese, and mixed-language text all raise different boundary decisions.

Takeaway: word tokenization is a useful starting point, not a universal definition of text units.

Words have parts

Morpheme : a minimal meaning-bearing unit in a language.

`fox`: one morpheme

`cats`: two morphemes `cat` and `-s`

Morphology : the study of morphemes

Morphemes in English and Chinese

Morphemes are meaningful pieces, but segmentation can still be hard.

Example	Possible morpheme analysis
worked	work + -ed
carefully	care + -ful + -ly
washing	wash + -ing
glasses	glass + -es
中文 examples	characters often correspond to morphemes, but not always a complete word segmentation

Morphemes are closer to meaning than characters, but they are not easy to identify automatically across languages.

Why not just use morphemes?

They are meaningful, but hard to define consistently.

Morphemes are attractive because they often carry meaning.

But for NLP systems, they are hard to use as a universal unit:

- boundaries are not always clear
- languages differ greatly in morphology
- automatic morphological analysis can itself be a difficult NLP task

So we still need a computationally robust unit.

Unicode

A standard for representing text across writing systems.

- Represents characters from many scripts used around the world.
- Includes letters, punctuation, currency symbols, mathematical symbols, and emoji.
- Separates the idea of a character from the bytes used to store it.

For NLP, this matters because “character” and “byte” are different computational units.

ASCII: some history for English

A narrow early encoding standard.

- ASCII uses 7 bits, giving 128 possible code points.
- Only a subset are printable characters; others were control characters.
- It was designed around English-centered computing and is far too limited for multilingual NLP.

Modern text processing requires Unicode and encodings such as UTF-8.

UTF-8 encoding

Characters can require different numbers of bytes.

Character range	Typical UTF-8 length
ASCII code points 0–127	1 byte
Many European, Middle Eastern, and African scripts	2 bytes
Many Chinese, Japanese, and Korean characters	3 bytes
Some rarer characters, symbols, and emoji	4 bytes

A byte-level tokenizer works below the character level.

Candidate units so far

Each choice has a different failure mode.

Unit	Strength	Problem
word	intuitive for many languages	ambiguous, huge vocabulary
morpheme	closer to meaning	hard to define automatically
character or byte	small vocabulary, high coverage	long sequences

The rest of the lecture asks how NLP systems compromise.

Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer(s),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

Language

Corpora vary by language.

- There are thousands of languages in the world.
- It is important to test NLP methods on multiple languages.
- A tokenization assumption that works for one language may fail for another.

Tokenization is not only a technical choice; it is also shaped by language and writing system.

Corpora vary along dimensions like

Variety , like African American English varieties

- AAE Twitter posts might include forms like " *iont*"
(*I don't*)

Genre: newswire, fiction, scientific articles, Wikipedia

Author Demographics : writer's age, gender, ethnicity, socio-economic status

There are simply too many words!

	Types = $ V $	Instances = N
Shakespeare	31 thousand	884,000
Brown Corpus	38 thousand	1 million
Switchboard conversations	20 thousand	2.4 million
COCA	2 million	440 million
Google N-grams	13+ million	1 trillion

The bigger the corpus, the more word types!

There are simply too many words

Vocabulary size grows as we see more text.

Heaps' Law is an empirical regularity:

$$|V| = K N^\beta, \quad \text{with } 0 < \beta < 1$$

- N = number of token occurrences in the corpus
- $|V|$ = number of distinct types
- As N grows, $|V|$ also keeps growing, though sublinearly.
- The midterm focus is the intuition, not fitting K or β .

Most word types are rare

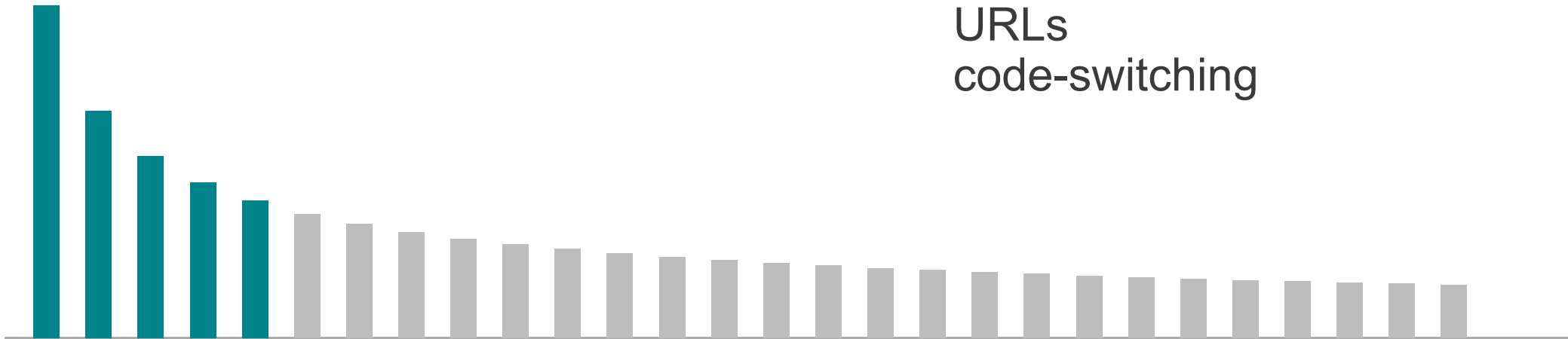
A fixed word vocabulary leaves a long tail.

a few very frequent words

the, of, and, to, is

many rare words

new names
technical terms
misspellings
URLs
code-switching



Frequency drops quickly, but the tail is very long.

Two kinds of words

Function words

- of, the, is, and, una, 是 ,

Content words

- mango, braise, snowy, feliz, 北京

Why is too many words a problem?

Fixed word vocabularies create out-of-vocabulary words.

If a system uses a fixed word vocabulary, then test text can contain words that were not in that vocabulary.

These are out-of-vocabulary words, or OOV words.

- new names
- technical terms
- misspellings
- new slang
- domain-specific forms

Words and Subwords

Because of these three problems:

1. Many languages don't have orthographic words
2. Defining words post hoc is challenging
3. The number of words grows without bound

Modern language models usually do not rely on

a fixed word-only vocabulary; they often use subwords or bytes.

Why tokenize?

A deterministic tokenization makes text processing well-defined.

- Systems can agree on the token sequence and token count for a text under the same tokenizer.
- Many NLP methods and evaluations assume a fixed tokenization before further processing.
- Subword and byte-based tokenizers greatly reduce the OOV problem.

Important caveat: comparisons across different tokenizers require care.

Before subwords: practical pre-tokenization

Many tokenizers start with simple rules.

Raw text:

Dr. Smith paid \$45.55 at 3 p.m.



rough chunks:

Dr. | Smith | paid | \$45.55 | at | 3 p.m. |

Small rules here can affect everything downstream.

Regular expressions are common tools

We introduce the idea, not detailed regex syntax.

- A regular expression is a formal pattern for matching strings.
- Regex rules are often used in preprocessing, formatting, and pre-tokenization.
- They are useful, but hand-written rules can be brittle.

Midterm level: understand why rule-based text processing can make false positives and false negatives.

Writing rules is iterative

A rule can be too narrow or too broad.

Goal	Find the word “the”
First rule	the
Problem	misses capitalized The
Broader rule	[tT]he
New problem	may match inside other strings, depending on boundary handling

Rule design often alternates between fixing missed matches and fixing incorrect matches.

False positives and false negatives

The process we just went through was based on **fixing two kinds of errors:**

1. Not matching things that we should have matched (The)

False negatives

2. Matching strings that we should not have matched (**the** re, **the** n, o**the** r)

False positives

Sentence Segmentation

!, ? mostly unambiguous but **period “.”** is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.

Issues for rule-based tokenization

Mostly but not always remove punctuation:

- [m.p.h.](#), [Ph.D.](#), [AT&T](#), [cap'n](#)
- prices ([\\$45.55](#))
- dates ([01/02/06](#))
- URLs (<http://www.stanford.edu>)
- hashtags ([#nlproc](#))
- email addresses (someone@cs.colorado.edu)

Numbers are tokenized differently across languages

- English [555,500.50](#) = French [555 500,50](#)

Multiword expressions (MWE)?

- [New York](#), [rock 'n' roll](#)

Word tokenization

The simplest idea is to use words as tokens.

Text: the movie was surprisingly good



Word tokens:



Good when words are clear. Fragile when the vocabulary is open-ended.

Word tokenization and OOV

A fixed word vocabulary cannot contain every possible word.

Word tokenization is simple when word boundaries are clear:

the movie was surprisingly good → the | movie | was | surprisingly | good

But unseen words create a problem:

Old strategy	Problem
Map unknown words to UNK	many different words collapse to one symbol
Hand-written normalization rules	can help, but cannot cover all cases

This motivates character, byte, and subword tokenization.

Character and byte tokenization

What if our basic units are smaller than words?

Character level

- one token is one Unicode character
- vocabulary depends on scripts
- example: 地 is one character

Byte level

- one token is one byte value
- vocabulary has 256 values
- any valid UTF-8 text can be represented

Tradeoff

Strong coverage and direct spelling information, but sequences become much longer.

This motivates a middle ground: subword tokenization.

Between words and characters: Subword Tokenization

How can we combine the efficiency of word tokenization with the high coverage of character tokenization?

Solution: *subword tokenization*, where tokens are words or parts of words

Tokenization with ChatGPT's tokenizer:



We'd sat back on the grass, and time
flew by as we looked at the Milky Way.

Intuition behind subword tokenization:

- Use data + algorithms to define what the vocabulary should be
- Subword vocabularies are learned from corpus statistics. BPE specifically uses frequencies of adjacent symbol pairs.
- While we sometimes get intuitive token segmentations (e.g., **We'd**), most of the time, these are not linguistically aligned (e.g., **Milky**, **looked**)

Common subword tokenization algorithms

We name three methods, but the midterm focuses on BPE.

Algorithm	Basic idea	In this course
Byte Pair Encoding (BPE)	start small and merge frequent adjacent pairs	main algorithm for this lecture
WordPiece	merge choices are based on a likelihood-style criterion	name only
Unigram LM tokenizer	start with many candidates and prune the vocabulary	name only

For the midterm: know BPE training and BPE encoding, not detailed WordPiece or Unigram derivations.

BPE: problem setup

What is learned, and from what?

Input	Output
training corpus D	vocabulary V
target number of merges	ordered merge list
initial units such as bytes	tokenizer for new text

Core rule: repeatedly merge the most frequent adjacent pair.

BPE: training algorithm

Count, merge, update, repeat.

Input: training corpus D, initial symbols, target vocabulary size or number of merges.

- Pre-tokenize if needed, for example by whitespace or another deterministic rule.
- Start from basic units, such as characters or bytes.
- Count adjacent symbol pairs in the current corpus representation.
- Merge the most frequent pair into a new symbol.
- Replace all occurrences of that pair and add the merge to the ordered merge list.
- Repeat until the target is reached.

The ordered merge list is what the encoder will use later.

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

Peter Piper picked
a peck of pickled
peppers

This is our
(very interesting)
training corpus D

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

Peter, _Piper,
_picked, _a, _peck,
_of, _pickled,
_peppers

First, we pre-tokenize
by splitting on
whitespace

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

Peter, _Piper,
_picked, _a, _peck,
_of, _pickled,
_peppers

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t}

Initialize a vocab of
individual symbols in D
as tokens

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t e r , _ P i p
e r , _ p i c k e
d , _ a , _ p e c k ,
_ o f , _ p i c k l
_ e d , _ p e p p e r
s

Convert D into a
sequence of tokens
(initial symbols)

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t}

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t e r , _ P i p
e r , _ p i c k e
d , _ a , _ p e c k ,
_ o f , _ p i c k l
e d , _ p e p p e r
s

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t}

Token Pair

Frequency

_ + p	4
p + e	4
c + k	3
e + r	3
e + d	2
i + c	2
p + i	2

Merge List

Initialize frequency
table of symbol pairs and
(empty) merge list

BPE step 1: choose a pair to merge

The pair-frequency table determines the next merge.

Token pair	Frequency
_ + p	4
p + e	4
c + k	3
e + r	3
e + d	2
i + c	2
p + i	2

There is a tie here.

A real tokenizer needs a deterministic tie-breaking rule. In this example we choose _ + p, then merge every occurrence of that pair.

Next merge: _ + p → _p

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t e r , _ P i p
e r , _ p i c k e d ,
_ a , _ p e c k , _ o
f , _ p i c k l e d ,
_ p e p p e r s

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t, _p}

Token Pair

Frequency

_ + p	4
p + e	4
c + k	3
e + r	3
e + d	2
i + c	2
p + i	2

Merge List

1. _ + p -> _p

Merge the token pair
into a new token

Update the corpus,
vocab, and merge list

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t e r , _ P i p
e r , _ p i c k e d ,
_ a , _ p e c k , _ o
f , _ p i c k l e d ,
_ p e p p e r s

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t, _p}

Token Pair

Frequency

c + k	3
e + r	3
_p + e	2
_p + i	2
e + d	2
i + c	2
p + e	2

Merge List

1. _ + p -> _p

Update the frequency
table (since the corpus
has changed)

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t e r , _ P i p
e r , _ p i c k e d ,
_ a , _ p e c k , _ o
f , _ p i c k l e d ,
_ p e p p e r s

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t, _p, ck}

Token Pair

Frequency

c + k	3
e + r	3
_p + e	2
_p + i	2
e + d	2
i + c	2
p + e	2

Merge List

1. _ + p -> _p
2. c + k -> ck

Merge the token pair
into a new token

Update the corpus,
vocab, and merge list

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t e r , _ P i p
e r , _ p i c k e d , _
a , _ p e c k , _ o f ,
_ p i c k l e d , _ p
e p p e r s

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t, _p, ck,
er}

Token Pair

Frequency

e + r	3
_p + e	2
_p + i	2
e + d	2
i + ck	2
p + e	2
_ + a	1

Merge List

1. _ + p -> _p
2. c + k -> ck
3. e + r -> er

Merge the token pair
into a new token

Update the corpus,
vocab, and merge list

What BPE is doing

It builds a vocabulary by compression-like merging.

At each step:

count adjacent pairs → choose most frequent → merge → update

What it learns:

- frequent words
- frequent word pieces
- sometimes pieces that are not linguistic units

BPE is data-driven, not meaning-driven.

BPE training: exam version

This is the procedure students should be able to reproduce.

Training algorithm:

1. Start from basic units, such as characters or bytes.
2. Count all adjacent token pairs in the training corpus.
3. Merge the most frequent pair into a new token.
4. Replace all occurrences of that pair.
5. Add the merge to the ordered merge list.
6. Repeat until the target vocabulary size or merge count is reached.

The merge list is ordered.

BPE encoding algorithm

After training, the merge rules are fixed.

Training produced an ordered merge list:

1. $n + e \rightarrow ne$ 2. $ne + w \rightarrow new$ 3. $_ + r \rightarrow _r$ 4. $_r + e \rightarrow _re$

- Start a new test word from the same basic units, such as characters or bytes.
- Apply the learned merges in the order learned during training.
- Do not learn new merges during encoding.
- Do not recount pair frequencies on the test text.

Training learns merge rules. Encoding applies those rules.

Encoding an unseen word

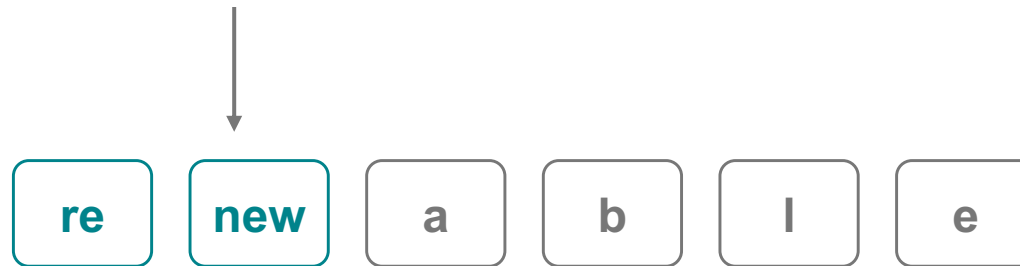
The point of subwords is graceful fallback.

Training words: new, renew, reset

Learned pieces: ne, new, re, reset

New word at test time: renewable

possible tokenization:



The full word was unseen, but its pieces are known.

BPE and bytes in GPT-style tokenizers

Byte-level BPE is common, but it is not the definition of BPE.

- BPE is an algorithm for merging symbols. The initial symbols can be characters or bytes.
- Many GPT-style tokenizers use byte-level BPE over UTF-8 encoded text.
- Byte-level tokenization starts from only 256 possible byte values.
- This gives complete coverage for valid UTF-8 text, though learned tokens may look unintuitive.

Key point: byte-level BPE is a common design choice, not the definition of BPE itself.

Visualizing GPT-4o tokens

Real tokenizer output often differs from visible word boundaries.

Example observations from a tokenizer visualizer:

- Common words may be single tokens, often with an initial space attached.
- Clitics such as 's can be split off in some words but not others.
- Numbers may be segmented into digit chunks rather than kept as one whole number.
- Preprocessing rules can affect punctuation, digits, and contractions.

Visible string	Possible token behavior
Jane's	Jane + 's
she's	possibly one frequent token
20260913	202 + 609 + 13
_anyhow vs anyhow	leading space can change the tokenization

Tokens are not the same as visible words

Spaces and punctuation can be part of the token inventory.

String	Possible token pieces
"hello"	hello
" hello"	_hello
"don't"	don + 't
"20260913"	202 + 609 + 13
"unbelievable"	un + believable or smaller pieces

Tokenization is learned from data plus rules, not from our visual intuition.

Tokenizing across languages

Many modern tokenizers are multilingual.

Training data and vocabulary allocation can still be uneven across languages.

A tokenizer may represent some languages with more tokens per sentence than others.

This matters because token count affects cost and how much text fits in a context window.

Tokenization efficiency can differ across languages

One tokenizer, one example

English

Natural language processing is useful.

Fewer tokens in this tokenizer example

Spanish

El procesamiento del lenguaje natural es útil.

More tokens in this tokenizer example

Claim we can make: tokenization efficiency can differ across languages and tokenizers.

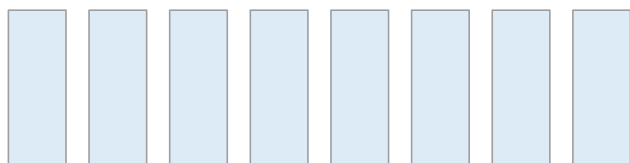
Claim we should not make: English is always tokenized better.

Why token count matters

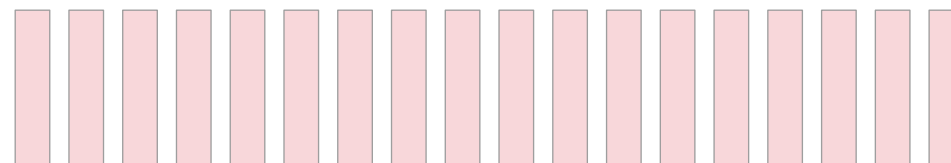
A fixed context window is measured in tokens, not words.

Same meaning, different token lengths

shorter tokenization



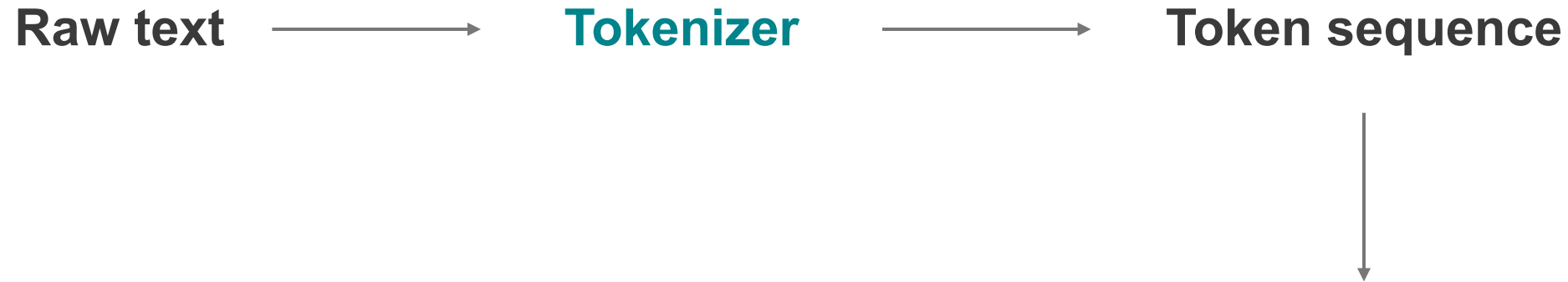
longer tokenization



In a fixed token budget, longer tokenization leaves less room for content.

Tokenization affects everything downstream

It happens before the model sees the text.



Later lectures: embeddings, language models, sequence labeling, neural networks, transformers

Today's lecture is the input layer of the course.

What we have not learned yet

Keeping the prerequisite chain clean.

Not today:

- word meaning as vectors
- language model probabilities
- hidden states
- logits and softmax
- neural network training

Today:

We learned how text becomes a sequence of units.

Midterm checkpoint: concepts

You should be able to define these precisely:

token

type

vocabulary

corpus

morpheme

OOV

subword

character vs byte

BPE

You should also be able to explain:

why word is difficult to define as a universal computational unit

why tokenization depends on language, genre, and task

why confusing token occurrences with vocabulary types is a common error

Exam principle: questions should target what was explicitly taught and practiced.

Midterm checkpoint: comparisons

You should be able to explain the tradeoffs.

Comparison	What to explain
word vs character	vocabulary size, coverage, sequence length
word vs subword	OOV and rare words
character or byte vs subword	efficiency and larger reusable units
rule based vs learned tokenization	hand written rules vs corpus statistics
BPE training vs BPE encoding	learn merges vs apply learned merges

Midterm checkpoint: BPE

You should be able to run a few merge steps by hand.

Training

- count adjacent pairs
- merge the most frequent pair
- update corpus and vocabulary
- repeat

Encoding

- start from basic units
- apply learned merges in learned order
- do not recount frequencies on the test text

If pair counts tie, the exam will explicitly state a tie rule.

Practice problem

A short version of the kind of BPE calculation that can appear on the exam.

word	frequency
low	5
lower	2
newest	6
widest	3

Tie rule

If several pairs have the same maximum frequency, choose the first pair encountered when scanning the corpus in the order shown, left to right within each word.

Tasks

- start from characters plus </w>
- count adjacent pairs
- perform the first two merges
- show the intermediate corpus after each merge
- encode the new word lowest using the learned ordered merges

Next: tokens and meaning

Lecture 3 begins after this lecture, not before it.

Today: **text → tokens**

Next: **tokens → meaning and prediction**

A tokenizer may split related-looking strings into overlapping pieces, but it does not understand meaning.

How can a model learn that “good” and “great” are related?

Lecture 3: Language Models and Embeddings